UNIVERSITY OF KELANIYA / SENG 34213
FOREVER HOTEL MANAGEMENT SYSTEM

Performance Test Results

Six-Subsystem Functional and Load-Test Evidence

Reporting date: 9 November 2026. Evidence is identified by subsystem repository, branch, commit SHA, and GitHub Actions run date; comparisons across separate repository commits do not constitute a single deployed six-subsystem release test.

Contents

1. Performance measurement status
2. Hotel Website (HW)
3. Manager Dashboard (MAD)
4. Front Desk System (FDS)
5. Food Ordering and Service Requests (FOSS)
6. Kitchen Management System (KMS)
7. Worker Management System (WKMS)
8. Integrated performance test protocol

1. Performance measurement status

As of 9 November 2026, no integrated six-subsystem staging load test (such as the 100-concurrent-user SRS NFR-01 benchmark or the 300-concurrent-user NFR-14 system-wide load test) has been executed against deployed multi-service infrastructure. Consequently, hotel-wide performance acceptance remains NOT VERIFIED. However, individual repositories now provide bounded local benchmark evidence (FDS 1,800-request local API benchmark) and verified load-test harnesses (HW performance runner, MAD 100/300-user harness, and API Gateway 100/300-socket smoke harness), which are recorded below without overclaiming system-wide SLA compliance.

Subsystem	Dedicated executed benchmark	Recorded p95 / throughput / errors	Status
HW	Load runner verified on local HTTP fixtures (8/8 pass, 7.373s); 300-user staging run not executed	Fixture tool test only; no deployed application latency/RPS recorded	TOOLING VERIFIED / STAGING NOT EXECUTED
MAD (+ Gateway)	MAD 100/300-user harness & API Gateway 100/300-socket smoke suite present; staging run unverified	Gateway smoke thresholds (p95 <=500ms, fanout <=2s) defined; no staging raw output	HARNESS PRESENT / NOT VERIFIED
FDS	Local read-only HTTP benchmark executed (5 GET endpoints x concurrency 1, 5, 10; 1,800 requests)	15/15 cases PASS; 0 errors (0%); max p95 153.23 ms; max throughput 1,016.85 req/s	LOCAL READ BENCHMARK EXECUTED
FOSS	Isolated local mock benchmark executed (8 scenarios SC-PERF-01..08; 1,490 requests; PR #120)	8/8 PASS; 0% errors (0.00%); max p95 484.3 ms; aggregate throughput 263.46 req/s	LOCAL BENCHMARK EXECUTED
KMS	No load or stress benchmark executed (665 functional CI tests pass)	Not available	NOT VERIFIED
WKMS	No load or stress benchmark executed (649 functional CI tests pass)	Not available	NOT VERIFIED

Relevant performance acceptance targets

SRS NFR-01 specifies that 95% of API requests should complete within 500 ms under 100 concurrent users, NFR-03 targets sub-second to 2-second real-time event propagation, and NFR-14 targets 300 concurrent guest/staff sessions without memory exhaustion or overbooking. Local single-service read benchmarks (such as FDS localhost GET measurements) and fixture-backed runner unit tests do not substitute for a coordinated staging benchmark across all services, API Gateway, RabbitMQ, and PostgreSQL.

2. Hotel Website (HW)

Repository: forever-hotel/hotel-website | Branch: develop | Tested revision: 5e3cd6bf323bc005e9efb86b5e09ba0e3015e727 (QA deliverable snapshot: 63673ba704b4ae645a8f067e246309b6fb0f03d0) | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/hotel-website/actions/runs/42504096480, https://github.com/forever-hotel/hotel-website/actions/runs/42504096452, documents/performance-report.pdf (commit 63673ba704)

The HW repository includes a closed-loop load runner (performance/run.mjs and performance/metrics.mjs) supporting up to 300 virtual guests with precreated accounts, photo cold/warm pair fetching, availability/quote refresh, unpaid reservation creation/cleanup, and a two-guest last-room race check. On 25 October 2026 (commit 63673ba704), npm run test:coverage in performance/ executed 8 disposable HTTP fixture scenarios in 7.373s (100% coverage on metrics.mjs; 94.34% lines / 90.30% branches / 100% functions on run.mjs). In addition, backend PostgreSQL CI (Run 42504096480) executes backend/test/integration/booking-race.integration-spec.ts against a disposable PostgreSQL instance. No 300-user staging load run has been executed.

Documented performance scenarios and tooling checks

Scenario	Operation / path	Required measurements / objective	Current status
HW-TOOL-01	performance/test/runner.test.mjs (percentiles & faults)	Verify p50/p95/p99 calculation and failure detection for timeout, imageFailure, overbooking, cleanupFailure, malformedReservation	PASS (local fixture)
HW-TOOL-02	performance/test/runner.test.mjs (mixed load & race)	Mixed HTTP load fixture with competing last-room reservation winner and credential-safe cleanup	PASS (local fixture)
HW-SQL-RACE	backend/test/integration/booking-race.integration-spec.ts	Two-guest concurrent final-room reservation lock verification on disposable CI PostgreSQL	PASS (CI PostgreSQL)
HW-P01..P04	Public catalogue, availability, auth, and photo cold/warm load	Staging p50/p95/p99 latency, RPS, error rate, and photo payload ratio under 300 virtual guests	Not executed on staging
HW-P05..P06	Staging NFR-14 300-user endurance & DB outage recovery time	Coordinated 300-user staging run with agreed NFR-14 p95/error budget and DB recovery latency	Not executed on staging

Measured outcomes

Required result	Current evidence
Tooling verification (performance/run.mjs & metrics.mjs)	8/8 fixture scenarios PASS in 7.373s (metrics.mjs 100% lines/branches; run.mjs 94.34% lines, 90.30% branches)
Staging URL, hardware, and NFR-14 numeric budget	Not supplied; example config deliberately rejects null targets
Deployed p50 / p95 / p99 and throughput (RPS)	Not measured against deployed staging application
Real SQL last-room contention	Verified for 2 competing guests in disposable PostgreSQL CI; not measured under 300-user staging load
SRS NFR-14 pass/fail determination	Undetermined (staging execution protocol pending)

3. Manager Dashboard (MAD)

Repository: forever-hotel/manager-dashboard (+ forever-hotel/api-gateway & forever-hotel/websocket-service) | Branch: development / develop | Tested revision: 8b2c63f459a93c217eb7cb502a2500e80181aa6e (api-gateway: f12aad6cb4; websocket-service: f12ef41ba9) | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/manager-dashboard/actions/runs/42480400290, https://github.com/forever-hotel/api-gateway/actions/runs/42504067779, https://github.com/forever-hotel/websocket-service/actions/runs/42504249365

Manager Dashboard defines backend/test/performance/run.ts, metrics.ts, and evaluate.ts for 100-user baseline and 300-user capacity comparison across 13 read-only management endpoints and Socket.IO sessions (5-minute warm-up, 15-minute measurement). Additionally, forever-hotel/api-gateway (commit f12aad6cb4) includes a dedicated performance smoke script (npm run test:performance) targeting 100 and 300 persistent WebSocket connections plus 1 HTTP request/client/s with thresholds of p95 <= 500 ms, 0 HTTP/socket errors, and fanout delivery <= 2,000 ms. Live multi-service staging benchmark outputs have not yet been archived.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
MAD-P01	100 authenticated-user baseline	13 read-only API routes and socket sessions; 5-min warm-up then 15-min measurement (p95 <= 500ms, 0 errors)	Harness verified / staging not run
MAD-P02	300 authenticated-user capacity comparison	Compare 300-user p95 against 1.1x 100-user baseline with 0 disconnects	Harness verified / staging not run
MAD-P03..04	Realtime event propagation & browser render	Commit-to-client delivery (<=1s) and browser UI visibility (<=2s) under synchronized clocks	Not tested on staging
GW-PERF-01	API Gateway 100 & 300 client smoke harness	100 & 300 persistent sockets + 1 req/client/s; p95 <= 500ms, 0 errors, fanout <= 2s	Script in api-gateway / staging pending

Measured outcomes

Required result	Current evidence
Harness & evaluator unit verification	Passing in MAD CI (Run 42480400290) and API Gateway CI (Run 42504067779)
Staging p50 / p95 / p99 and throughput (RPS)	Not recorded for deployed staging cluster
SRS NFR-01 / NFR-03 / NFR-14 pass/fail determination	Not evaluated (staging run required)

4. Front Desk System (FDS)

Repository: forever-hotel/front-desk | Branch: develop (4b5ef57d4c) & feature/84-fds-final-qa-evidence (1065617 / a6050dd) | Benchmark execution date: 27 October 2026 | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/front-desk/actions/runs/42506217082, evidence/fds-final-qa/performance/performance-results.json (commit a6050dd950217da0255fdb1d2fe678098694fd0c on application revision 106561705fe5d7a66e06fe1a7dd14f255b4ba744)

Unlike the 25 October 2026 consolidated report (when FDS had no executed benchmark), FDS executed a reproducible local read-only HTTP benchmark on 27 October 2026 via scripts/verification/fds-final-qa/run-fds-performance.cjs against a fresh disposable PostgreSQL 16 instance (localhost:55436) and local NestJS FDS backend (localhost:4308, Node.js v24.21.0, Windows 10). Five GET endpoints were tested across three concurrency levels (1, 5, and 10) using 12 warm-up requests and 120 measured requests per case (1,800 measured requests total). All 15 cases passed with 0 HTTP errors (0.00%), a maximum case p95 of 153.23 ms, and peak throughput of 1,016.85 requests/s.

Executed FDS local read-only benchmark results (1,800 measured requests)

Case ID	Endpoint / operation	Conc. / N	Observed p50 / p95 / RPS / Errors
PERF-01-C1 / C5 / C10	GET /health/ready (Database readiness)	1 / 5 / 10 (120 ea)	p50: 3.89 / 8.29 / 20.80 ms | p95: 10.30 / 32.93 / 41.77 ms | RPS: 177.09 / 393.07 / 347.52 | 0% err (PASS)
PERF-02-C1 / C5 / C10	GET /rooms/status (Room status board)	1 / 5 / 10 (120 ea)	p50: 5.64 / 10.19 / 21.44 ms | p95: 14.38 / 30.21 / 44.89 ms | RPS: 129.74 / 384.83 / 418.67 | 0% err (PASS)
PERF-03-C1 / C5 / C10	GET /bookings/recent?limit=5 (Recent bookings)	1 / 5 / 10 (120 ea)	p50: 4.17 / 9.71 / 64.92 ms | p95: 8.77 / 16.76 / 153.23 ms | RPS: 180.79 / 478.06 / 136.87 | 0% err (PASS)
PERF-04-C1 / C5 / C10	GET /bookings/search?query=CI (Booking search)	1 / 5 / 10 (120 ea)	p50: 4.93 / 9.35 / 18.95 ms | p95: 13.90 / 23.73 / 58.15 ms | RPS: 148.36 / 444.58 / 426.10 | 0% err (PASS)
PERF-05-C1 / C5 / C10	GET /realtime/contract (Realtime contract)	1 / 5 / 10 (120 ea)	p50: 1.67 / 4.39 / 12.28 ms | p95: 4.87 / 8.30 / 23.54 ms | RPS: 466.30 / 1,016.85 / 679.97 | 0% err (PASS)

Measured outcomes and boundaries

Required result	Current evidence
Test environment, dataset, and commit	Localhost NestJS + disposable PostgreSQL 16; deterministic synthetic fixture; revision 1065617 (evidence commit a6050dd)
Concurrency profile and total requests	Concurrency 1, 5, 10; 12 warm-up + 120 measured requests per case = 1,800 measured requests across 15 cases
Observed p50 / p95 and throughput	All 15/15 cases PASS; p50 1.67–64.92 ms; highest case p95 153.23 ms (PERF-03-C10); peak RPS 1,016.85 req/s (PERF-05-C5)
Failed requests / timeout rate	0 HTTP errors (0.00%) across 1,800 measured requests (timeout threshold 7,000 ms)
Unmeasured FDS scopes	Write-load (walk-in, check-in, room change, checkout), Socket.IO fan-out latency, RabbitMQ-to-WKMS delay, and 100-user staging load remain unmeasured

5. Food Ordering and Service Requests (FOSS)

Repository: forever-hotel/food-ordering | Branch: develop | Tested revision: c786e42b26b3a0f7966eb85c6df2b591b72e90e7 (QA deliverable snapshot: 9ac1e6f3df9f93ee76be6db479dc8c4bf6529328) | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/food-ordering/pull/120, https://github.com/forever-hotel/food-ordering/actions/runs/42644489101, backend/test/perf/benchmark.spec.ts, backend/test/perf/benchmark-results.json

The committed FOSS QA benchmark suite (backend/test/perf/benchmark.spec.ts, PR #120 under Refs #119 / DDP-274) executes 8 automated concurrency scenarios against the untouched production codebase in an isolated container (127.0.0.1, zero remote DB access). Across 1,490 HTTP requests, all 8 scenarios achieved 0 errors (0.00%) and met SRS NFR-01 (all p95 < 500 ms; p50 = 48.9-302.5 ms, p95 = 70.5-484.3 ms) under concurrency 25-100 VUs. Aggregate throughput was 263.46 req/s (1,490 reqs / 5.66s). Integrated staging load tests remain unverified.

Documented performance scenarios (FOSS executed benchmark)

Scenario	Operation / path	Required measurements / objective	Current status
SC-PERF-01	POST /guest-sessions/qr (QR activation)	25 VUs, 200 reqs; p50: 48.9 ms, p95: 70.5 ms; 504.5 req/s (gate <300ms)	PASS (0% err)
SC-PERF-02	GET /menu & /menu/windows (Menu browse)	50 VUs, 200 reqs; p50: 302.5 ms, p95: 484.3 ms; 102.0 req/s (gate <500ms)	PASS (0% err)
SC-PERF-03	POST /food-orders (Order burst)	25 VUs, 150 reqs; p50: 78.1 ms, p95: 210.9 ms; 254.0 req/s (gate <500ms)	PASS (0% err)
SC-PERF-04	GET /food-orders (Order history)	80 VUs, 240 reqs; p50: 116.5 ms, p95: 155.6 ms; 697.0 req/s (gate <500ms)	PASS (0% err)
SC-PERF-05	POST /service-requests (Service request)	25 VUs, 150 reqs; p50: 55.6 ms, p95: 86.7 ms; 422.7 req/s (gate <500ms)	PASS (0% err)
SC-PERF-06	POST /complaints (Complaint submit)	25 VUs, 150 reqs; p50: 73.6 ms, p95: 93.3 ms; 355.7 req/s (gate <500ms)	PASS (0% err)
SC-PERF-07	GET /realtime/events/poll (Realtime SSE)	100 VUs, 300 reqs; p50: 93.3 ms, p95: 140.6 ms; 916.8 req/s (gate <500ms)	PASS (0% err)
SC-PERF-08	GET /menu (In-memory cache read)	25 VUs, 100 reqs; p50: 216.6 ms, p95: 473.4 ms; 79.4 req/s (gate <500ms)	PASS (0% err)

Measured outcomes

Required result	Current evidence
Automated benchmark suite execution (c786e42 / PR #120)	1,490 requests across 8 automated scenarios PASS with 0 errors (0.00% error rate; 100% 2xx status codes)
Throughput and latency distribution (empirical)	p50 = 48.9-302.5 ms, p95 = 70.5-484.3 ms (max p95 484.3 ms < 500 ms NFR-01 gate); aggregate throughput 263.46 req/s
SRS NFR-01 / NFR-03 / NFR-14 pass/fail determination	LOCAL BENCHMARK PASSED (all p95 < 500 ms at up to 100 VUs); multi-service staging cluster load testing pending

6. Kitchen Management System (KMS)

Repository: forever-hotel/kitchen-management | Branch: develop | Tested revision: 2bfa14b5535c5c065f49dbdc2b7c62c4c8105d15 | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/kitchen-management/actions/runs/42502180570

KMS functional CI coverage expanded to 665 passing tests across 123 suites (54 backend unit suites, 10 backend E2E suites covering order creation/reading/transitions, allergy safety, ordering windows, and menu management, plus 59 frontend Jest suites). However, no load, stress, KOT queue throughput, or cross-service FOSS-KMS-WKMS latency benchmark has been executed.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
KMS-P01	Menu read & category queries	Category/menu list p50/p95/p99, error rates, and DB query time	Not tested
KMS-P02	Incoming food-order burst	Order creation throughput, duplicate rejection, and DB lock performance	Not tested
KMS-P03	Concurrent kitchen status transitions	Concurrent chef status updates, atomicity, and outbound event emission	Not tested
KMS-P04	KOT print queue throughput	Print queue latency, retry handling, and duplicate prevention	Not tested
KMS-P05..06	FOSS-KMS-WKMS propagation & outage recovery	End-to-end timestamped event propagation and DB/broker recovery time	Not tested

Measured outcomes

Required result	Current evidence
Test environment, concurrency, and latency percentiles	Not measured (functional CI only: 665 passed, 6 skipped in Run 42502180570)
SRS pass/fail determination	Not evaluated

7. Worker Management System (WKMS)

Repository: forever-hotel/worker-management | Branch: develop | Tested revision: 1a33059c0005752ca709ca91ab11067dc6cff55a | CI date: 9 November 2026

Evidence: https://github.com/forever-hotel/worker-management/actions/runs/42504698971, https://github.com/forever-hotel/worker-management/actions/runs/42504970394

WKMS passes 649 automated unit/component tests (400 backend, 249 frontend) with >80% whole-application branch and line coverage on develop (1a33059c00), and local cross-service boundary checks with FDS (Phase B checkout consumer/escalation outbox and W04-B manual assignment) were demonstrated in bounded functional runs. Sustained load, concurrent task-claim race latency under 100–300 users, and Socket.IO fan-out benchmarks remain unexecuted.

Documented performance scenarios

Scenario	Operation / path	Required measurements / objective	Current status
WKMS-P01	Worker task queue reads (GET /wkms/tasks/queue)	p50/p95/p99 latency and error rate under concurrent shift load	Not tested
WKMS-P02	Task claim concurrency (POST /wkms/tasks/:id/claim)	Max-three-tasks rule and double-claim lock contention under load	Not tested
WKMS-P03	Worker start / complete transitions	Transactional throughput, DB lock waits, and error rate	Not tested
WKMS-P04..05	RabbitMQ checkout intake & Socket.IO broadcast	FDS-to-WKMS event ingestion and task.created/claimed/escalated fan-out latency	Not tested
WKMS-P06	Shift summary dashboard (GET /wkms/shift/summary)	Response-time distribution under concurrent supervisor queries	Not tested

Measured outcomes

Required result	Current evidence
Test environment, concurrency, and latency percentiles	Not measured (functional CI only: 649 passed, 43 skipped in Run 42504698971)
SRS pass/fail determination	Not evaluated

8. Integrated performance test protocol

1. Deploy a version-locked staging release across all six subsystem backends/frontends, forever-hotel/api-gateway, forever-hotel/websocket-service, disposable PostgreSQL 16, and RabbitMQ using forever-hotel/infra.

2. Provision synthetic guest, receptionist, chef, worker, and manager accounts, dedicated room inventory (including an isolated single-room stay for HW last-room contention), and sandboxed payment/email/print stubs.

3. Record exact commit SHAs, container digests, CPU/RAM limits, PostgreSQL pool sizes, and dataset cardinalities prior to execution.

4. Execute the checked-in harnesses (HW performance/run.mjs, MAD backend/test/performance/run.ts, API Gateway npm run test:performance, FDS run-fds-performance.cjs extended with write flows, and FOSS k6 scenarios SC-PERF-01..08) at graduated concurrency (1, 10, 50, 100, and 300 virtual users).

5. Archive raw JSON/CSV latency distributions (p50, p95, p99), throughput (RPS), HTTP/socket error rates, RabbitMQ consumer lag, and synchronized end-to-end event propagation traces before claiming NFR-01, NFR-03, or NFR-14 compliance.